Mandatory 4 – Ricart-Agrawala

Frederik Groth Termansen & Aske Thomas Neye

Github Repo:

https://github.com/askeneye/Mandatory4-ricartagrawala

System requirements

R1: System specification

Each node is a separate Go process with its own gRPC server and one gRPC client per peer. The service RAService has two unary RPCs:

RequestCS(Request) carries the sender's Lamport timestamp and ID. It asks for permission and **SendReply(Reply)** carries the sender's timestamp, the sender and receiver IDs, and the timestamp of the request it answers. It grants permission.

Service discovery is hardcoded, every node holds the same map from ID to address (localhost:5051, incrementing with one to localhost:5054). Every node connects to every entry except itself and listens on its own address. Clients are created lazily with grpc.NewClient, and request are sent with grpc.WaitForReady(true), so the nodes can start in any order.

The critical section is emulated. A node that enters logs ">>> ENTER critical section", then sleeps for one second and logs "<<< EXIT critical section". Any node may request access, each runs a loop that waits for 5 seconds, request access, runs the critical section and releases it.

Every node keeps an enum state (RELEASED, WANTED or HELD), a Lamport clock, the timestamp of its current request, a list of deferred requests and the set of peers that have replied. A mutex protects this state, because each incoming RPC runs on its own goroutine.

R2: Safety

A node enters the critical section only after every peer has replied to the nodes current request. Two nodes A and B can therefore not be in the critical section at the same time, because that would require each to have replied to the other while both were requesting.

1. When a node receives a request while it is the HELD state, it defers the reply.
2. When it receives a request while WANTED, it compares (timestamp, id) pairs. The smaller timestamp wins, and if the timestamps are equal, the smaller id wins. The winner defers its reply, so both node reach the same verdict and exactly one replies
3. Deferred replies are sent only in releaseAccess, after the node has left the critical section and its state is set to RELEASED.

SendReply also discards replies that do not belong to the current request, replies that arrive while the node is not WANTED, replies with a request_timestamp that differs from the node and duplicates from a peer that has already replied. Therefore, a late reply to an old request will not count towards a new one.

In the logs this is supported as between a node's ENTER and EXIT of the critical section, no other node logs ENTER.

R3: Liveness

Every request is eventually granted, because no reply is withheld forever. Consider waiting request with the smallest timestamp and id. Every other node is in either:

- RELEASED (replies at once)
- WANTED (with a larger pair, it replies at once)
- HELD (it leaves the critical section, and the sends all its deferred replies)

The node with the smallest request therefore receives all replies and enters. When it requests again, its new timestamp is larger than the requests it has already seen, because the receives advanced the nodes Lamport clock.

The argument assumes that the nodes do not crash and that the gRPC eventually delivers every message.

Algorithmic discussion

The run shown in figure 1, shows the nodes taking turns in the critical section, and deferred reply holds a waiting node back. Figure 1 comes from the same run as the logs in the appendix. It combines node 1 and 3.

```
17:29:53 [4]  Node 1: state WANTED, requesting CS
17:29:53 [11] Node 1: received reply from 3 (3/3)
17:29:53 [11] Node 1: state HELD, >>> ENTER critical section
17:29:53 [7]  Node 3: state WANTED, requesting CS
17:29:53 [12] Node 1: REQUEST from 3 (ts=7), state=HELD, ownTs=4,
ownPriority=true
17:29:53 [12] Node 1: deferring reply to 3
17:29:54 [12] Node 3: received reply from 4 (2/3)
17:29:54 [12] Node 1: <<< EXIT critical section
17:29:54  Node 1: state RELEASED, sending deferred replies
17:29:54 [14] Node 3: received reply from 1 (3/3)
17:29:54 [14] Node 3: state HELD, >>> ENTER critical section
```

Figure 1: Lines from node 1 and node 3 in the appendix run, merged and ordered by cause and effect (send before receive).

1. Node 1 requests with timestamp 4, receives 3 of 3 replies and enter the critical section.
2. Node 3 requests with timestamp 7. Node 1 is HELD, so it defers its reply to node 3.
3. Node 3 has replies from nodes 2 and 4 but is stuck at 2 of 3, because node 1's reply is missing. It cannot enter while node 1 is still inside.
4. Node 1 exits, sets its state to RELEASED and sends deferred reply. Node 3 gets 3 of 3 replies it changes state to HELD and enters the critical section.

The same pattern repeats afterwards with, node 3 defers another node while it is HELD, and the other node tries to enter, which it can when node 3 exits. In figure 1 we also see that no ENTER line appear between another nodes ENTER and EXIT lines, which would also repeat it the following patterns.

Each node updates its clock (Lamport clock) to max(local, received) + 1 on every received message and increments it on every send. That is why every request carries a timestamp that all nodes interpret the same way.

The cost of this operation is as following: one entry to the critical section takes 2(N-1) messages, one entry to the critical section takes N-1 replies and one entry to the critical section takes N-1 request. That is 6 messages per entry with four nodes.

Appendix: System logs

Node 1

17:29:48 Node 1: listening on localhost:5051 with 3 peers

17:29:51 [2] Node 1: REQUEST from 2 (ts=1), state=RELEASED, ownTs=0, ownPriority=true

17:29:51 [2] Node 1: granting permission to 2 for request 1

17:29:53 [4] Node 1: state WANTED, requesting CS

17:29:53 [9] Node 1: received reply from 2 (1/3)

17:29:53 [10] Node 1: received reply from 4 (2/3)

17:29:53 [11] Node 1: received reply from 3 (3/3)

17:29:53 [11] Node 1: state HELD, >>> ENTER critical section

17:29:53 [12] Node 1: REQUEST from 3 (ts=7), state=HELD, ownTs=4, ownPriority=true

17:29:53 [12] Node 1: deferring reply to 3

17:29:54 [12] Node 1: <<< EXIT critical section

17:29:54 Node 1: state RELEASED, sending deferred replies

17:29:54 [14] Node 1: REQUEST from 4 (ts=10), state=RELEASED, ownTs=4, ownPriority=true

17:29:54 [14] Node 1: granting permission to 4 for request 10

Node 2

17:29:36 Node 2: listening on localhost:5052 with 3 peers

17:29:41 [1] Node 2: state WANTED, requesting CS

17:29:51 [4] Node 2: received reply from 3 (1/3)

17:29:51 [5] Node 2: received reply from 4 (2/3)

17:29:52 [6] Node 2: received reply from 1 (3/3)

17:29:52 [6] Node 2: state HELD, >>> ENTER critical section

17:29:53 [6] Node 2: <<< EXIT critical section

17:29:53 Node 2: state RELEASED, sending deferred replies

17:29:53 [7] Node 2: REQUEST from 1 (ts=4), state=RELEASED, ownTs=1, ownPriority=true

17:29:53 [7] Node 2: granting permission to 1 for request 4

17:29:53 [9] Node 2: REQUEST from 3 (ts=7), state=RELEASED, ownTs=1, ownPriority=true

17:29:53 [9] Node 2: granting permission to 3 for request 7

17:29:54 [11] Node 2: REQUEST from 4 (ts=10), state=RELEASED, ownTs=1, ownPriority=true

17:29:54 [11] Node 2: granting permission to 4 for request 10

Node 3

17:29:48 Node 3: listening on localhost:5053 with 3 peers

17:29:51 [2] Node 3: REQUEST from 2 (ts=1), state=RELEASED, ownTs=0, ownPriority=true

17:29:51 [2] Node 3: granting permission to 2 for request 1

17:29:53 [5] Node 3: REQUEST from 1 (ts=4), state=RELEASED, ownTs=0, ownPriority=true

17:29:53 [5] Node 3: granting permission to 1 for request 4

17:29:53 [7] Node 3: state WANTED, requesting CS

17:29:53 [11] Node 3: received reply from 2 (1/3)

17:29:54 [12] Node 3: received reply from 4 (2/3)

17:29:54 [14] Node 3: received reply from 1 (3/3)

17:29:54 [14] Node 3: state HELD, >>> ENTER critical section

17:29:54 [15] Node 3: REQUEST from 4 (ts=10), state=HELD, ownTs=7, ownPriority=true

17:29:54 [15] Node 3: deferring reply to 4

17:29:55 [15] Node 3: <<< EXIT critical section

17:29:55 Node 3: state RELEASED, sending deferred replies

Node 4

17:29:49 Node 4: listening on localhost:5054 with 3 peers

17:29:51 [2] Node 4: REQUEST from 2 (ts=1), state=RELEASED, ownTs=0, ownPriority=true

17:29:51 [2] Node 4: granting permission to 2 for request 1

17:29:53 [5] Node 4: REQUEST from 1 (ts=4), state=RELEASED, ownTs=0, ownPriority=true

17:29:53 [5] Node 4: granting permission to 1 for request 4

17:29:53 [8] Node 4: REQUEST from 3 (ts=7), state=RELEASED, ownTs=0, ownPriority=true

17:29:53 [8] Node 4: granting permission to 3 for request 7

17:29:54 [10] Node 4: state WANTED, requesting CS

17:29:54 [16] Node 4: received reply from 1 (1/3)

17:29:54 [17] Node 4: received reply from 2 (2/3)

17:29:55 [18] Node 4: received reply from 3 (3/3)

17:29:55 [18] Node 4: state HELD, >>> ENTER critical section

17:29:56 [18] Node 4: <<< EXIT critical section

17:29:56 Node 4: state RELEASED, sending deferred replies